

Project

- *와인 스펙리에 AI 챗봇 만들기*

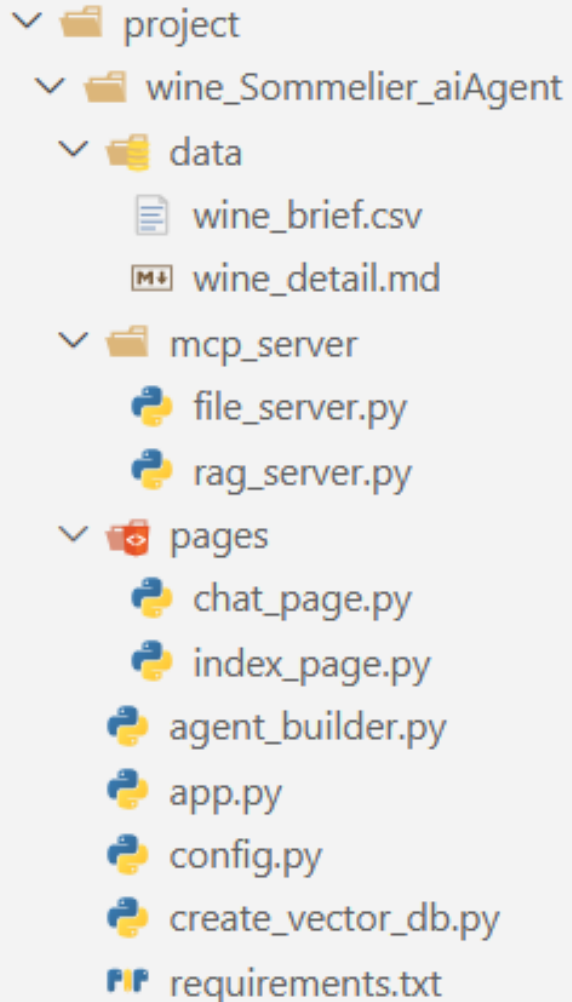
1. 서비스 기획 및 아키텍처

1. 시스템 아키텍처

- 구축할 시스템의 전체적인 구조
 - 하나의 에이전트(Client)가 여러 개의 서버(Server)를 거느리는 형태
 - 시스템은 크게 세 가지 계층(Layer)으로 나뉨
 1. 사용자 인터페이스: 사용자와 대화하는 창구. Streamlit을 사용하여 웹 브라우저에서 실행되며, 사용자의 질문을 에이전트에 전달하고 답변을 화면에 표시
 2. 에이전트 계층: 시스템의 두뇌. 랭체인으로 구현된 에이전트가 위치하며, MultiServerMCPClient를 통해 여러 도구 서버와 연결. 사용자의 질문을 분석하여 어떤 도구를 사용할지 판단
 3. 도구 계층: 실제 작업을 수행하는 손과 발. 독립적인 프로세스로 실행되는 MCP 서버와 도구들로 구성
 - YouTube Tool: 관련 와인 리뷰 영상 검색
 - File Server: 상담 내용을 기록하거나 파일을 생성
 - RAG Server: 와인 상세 설명 문서와 가격 정보 검색
 4. 대화 맥락 관리: 에이전트가 사용자와의 대화를 기억하고, 이전 맥락을 기준으로 답변하도록 하려면 대화 내용을 저장할 저장소 필요. 이를 위해 비동기 처리가 가능한 AsyncSqliteSaver를 사용하여 대화 내역을 관리

1. 서비스 기획 및 아키텍처

2. 프로젝트 폴더 구조



3. 데이터셋 준비

1) 정형 데이터

- 'wine_brief.csv' 는 와인의 이름, 와인 빈티지, 지역, 가격, 재고량 같은 수치 정보가 담긴 csv 파일
- 사용자가 "5만원 이하의 레드 와인 찾아줘"라고 물었을 때, 에이전트가 정확한 필터링을 수행하기 위해 사용

2) 비정형 데이터

- 'wine_detail.md' 는 각 와인의 특징, 어울리는 음식(페어링), 테이스팅 노트 등이 줄글 형태로 작성된 마크다운 파일.
- "이 와인의 맛 특징을 자세히 설명해 줘" 같은 질문에 풍성한 답변을 제공하기 위해 사용

4. 정형/비정형 이 두 데이터를 하나의 벡터 저장소에 담되, 메타데이터 정보를 토대로 필터링 처리한 RAG 시스템을 구축할 것. 이렇게 하면 에이전트는 '가격 비교'라는 논리적인 작업과 '정보 검색'이라는 언어적인 작업을 명확히 구분하여 수행할 수 있게 됨

2. 프로젝트 환경 및 기본 UI 구성

1. 프로젝트 생성 및 환경 설정

- 1) 가상 환경 생성과 Streamlit 설치
- 2) 오픈AI API 키 작성

2. Streamlit 기본 구조 만들기

- Streamlit 앱의 기본적인 화면 구성
 - 1) app.py: 앱 구동을 위한 파일
 - 2) pages/index_page.py: 최초 접속 페이지
 - 3) pages/chat_page.py: 챗봇 기능 구현
 - 4) 실행 및 확인

3. 랭체인 에이전트 설계

1. 에이전트 설정 분리

- config.py: 에이전트의 성격과 기본 설정을 정의

2. 에이전트 생성기 작성

- agent_builder.py: 실제 에이전트 객체를 생성
 - 에이전트 생성 함수 작성
 - 비동기 메모리 도입
 - 비동기 에이전트 실행 함수 정의

3. 에이전트 동작 확인 및 UI 연결

- 이벤트 루프(event loop) 관리
- 기본 설정과 세션 상태 초기화
- 사용자 입력 처리
- 동작 확인

4. 하이브리드 지식 저장소 구축

- 데이터 전처리 및 벡터 스토어 생성 코드 작성

- 두 개의 데이터(wine_brief.csv, wine_detail.md)를 하나의 벡터 저장소에 통합
 - 검색 대상 기준을 정함
 - 와인의 상세 설명(마크다운 파일)과 주요 특징을 텍스트로 합쳐서 저장
 - 메타데이터로 필터링 기준 지정
 - source_type과 category를 설정하여 와인에 대한 설명인지, 가격에 대한 정보인지를 구분하여 저장
 - 에이전트가 '5만원 이하'라는 조건을 걸었을 때 이 값을 기준으로 필터링
- 'create_vector_db.py' 파일 생성
 - 데이터 전처리 작업과 데이터베이스 생성 작업을 모두 진행
 - 벡터 스토어 생성

5. MCP 서버를 통한 도구 확장

- 에이전트에게 전달할 벡터 데이터베이스 구성이 끝났으면, 이제 이 데이터베이스를 사용할 수 있는 도구를 만들어 줄 차례
- 여기서는 2개의 독립적인 MCP 서버를 만들어 운용함
 - File Server : 상담 결과를 리포트로 저장하는 '기록원' 역할
 - RAG Server : 와인 정보를 검색하고 조회하는 '사서' 역할
- 이 서버들은 에이전트와 별도의 프로세스로 실행되며, 필요할 때마다 호출되어 에이전트의 외부 MCP 서버 도구가 되어줄 것임

5. MCP 서버를 통한 도구 확장

1. 파일 관리 서버

- 에이전트가 적성한 텍스트를 파일로 저장해 주는 역할
- 서비스의 안정성을 위해 '샌드박스(Sandbox)' 개념 도입
- 에이전트가 시스템의 아무 파일이나 건드리지 못하도록, output이라는 전용 폴더를 만들고 그 안에서만 파일이 생성되도록 제한을 둘 것임
- MCP 서버들은 하나의 mcp_server 폴더에서 관리
- write_markdown 도구 정의
- MCP 서버 설정
- MCP 서버 부착
- 동작 확인

5. MCP 서버를 통한 도구 확장

2. RAG 서버

- 에이전트에게 지식을 공급해 줄 와인 검색 서버
- 벡터 스토어를 불러와, 에이전트가 질문을 던지면 관련 정보를 찾아주는 역할
- RAG(검색 증강 생성)의 핵심인 retriever 객체를 만들어 검색 기능을 구현한 도구가 포함된 서버를 만들
- mcp_server 폴더에 'rag_server.py' 파일을 생성하고, 기본 설정 작성
- search_wine 도구 정의
- MCP 서버 실행 코드 작성
- 동작 확인

3. 영상 검색 도구 추가

- YoutubeSearchTool를 사용한 검색 도구를 에이전트에 추가
- 동작 확인